

Problem A. Sequence and Sequence

Let $P(i)$ be the sequence in which value p appears $p + 1$ times:

$$1, 1, 2, 2, 2, 3, 3, 3, 3, \dots$$

Equivalently, $P(n)$ is the minimum p satisfying $p(p + 3)/2 \geq n$. We want to compute the recursively defined sequence Q , and the useful convention is $P(0) = Q(0) = 0$ while the very small values are precomputed directly.

Although n can be huge, $P(i)$ has only $\mathcal{O}(\sqrt{n})$ distinct values, and only $\mathcal{O}(\sqrt{n})$ positions satisfy $P(i) \neq P(i - 1)$. Therefore the goal is to rewrite sums in terms of differences $Q(P(i)) - Q(P(i - 1))$.

First,

$$Q(n) = \sum_{i=1}^n Q(P(i)).$$

Set $f^{(0)}(x) = 1$ and $F^{(0)}(x) = \sum_{i=1}^x f^{(0)}(i)$. Abel summation gives

$$Q(n) = F^{(0)}(n)Q(P(n)) - \sum_{i=1}^n F^{(0)}(i - 1) (Q(P(i)) - Q(P(i - 1))).$$

Only the positions where P changes remain, so the second sum becomes

$$\sum_{i=1}^{P(n)} F^{(0)}\left(\frac{i(i + 1)}{2} - 1\right) Q(P(i)).$$

Thus the same problem reappears on input $P(n) = \mathcal{O}(\sqrt{n})$.

Define

$$f^{(d+1)}(x) = F^{(d)}\left(\frac{x(x + 1)}{2} - 1\right),$$

$$F^{(d)}(x) = \sum_{i=1}^x f^{(d)}(i),$$

$$g(d, n) = \sum_{i=1}^n f^{(d)}(i)Q(P(i)).$$

Then

$$g(d, n) = F^{(d)}(n)g(0, P(n)) - g(d + 1, P(n)),$$

and $g(0, n) = Q(n)$. After about four recursive levels, the input size is small enough to answer from a table.

The functions $f^{(d)}$ and $F^{(d)}$ are polynomials. Each level changes the degree from r to about $2(r + 1)$, so with depth $D = 4$ the maximum degree is only $K = 30$. Store polynomials by their values at integer points and evaluate them by Lagrange interpolation. With the constants used in the official solution, preprocessing takes $\mathcal{O}(K^2 + DK + DN)$, and each test case takes $\mathcal{O}(2^{D+1}K)$.

Problem B. Kawa Exam

Model exam questions as vertices and bugs as edges. Inside one connected component all vertices must choose the same answer, so without deleting any edge the score of a component is the frequency of its most common answer.

Deleting a non-bridge edge does not change the connected components, hence the score is unchanged. Deleting a bridge splits one tree component into a subtree and its complement. If the original component has mode frequency M , and the two new parts have mode frequencies M_1 and M_2 , then the new total score changes by

$$-M + M_1 + M_2.$$

Use Tarjan's algorithm to compress edge-biconnected components. Each compressed connected component is a tree, and every original bridge corresponds to one tree edge. The original values inside each compressed node are stored as a multiset contribution of that node.

For every tree edge, we need the mode frequency inside the child subtree and outside that subtree. Compute subtree modes by DSU on tree. At the same time, maintain an outside multiset initialized with all values of the component; when the current subtree is added to the inside structure, remove the same values from the outside structure.

To update a mode in constant time, store $cnt[x]$, the current frequency of value x , and $bucket[c]$, the number of values whose current frequency is c . Insertion increases $cnt[x]$ and updates the best frequency by taking a maximum. Deletion only needs to decrease the best frequency while $bucket[best] = 0$.

Keep the original edge indices on the compressed tree edges so that answers can be written back to the original bug order. $\mathcal{O}(n \log n)$ time is sufficient with DSU-on-tree merging.

Problem C. Flippy Sequence

Scan the two strings and split all positions where $s_i \neq t_i$ into maximal mismatch intervals. Two interval flips can create at most two mismatch intervals, so if there are three or more, the answer is 0.

There are three remaining cases.

If there is no mismatch interval, then the two operations must flip the same nonempty interval. The number of choices is

$$\frac{n(n+1)}{2}.$$

If there is one mismatch interval of length ℓ , there are two types of constructions. Two disjoint intervals may have union equal to the target interval, giving $2(\ell-1)$ ordered choices. Or two overlapping intervals may have difference equal to the target interval, giving $2(n-\ell)$ choices. The total is

$$2(\ell-1) + 2(n-\ell) = 2(n-1).$$

If there are two mismatch intervals, there are three structural ways to realize them, and the two operations are ordered, so the answer is $2 \cdot 3 = 6$.

The time complexity is $\mathcal{O}(n)$.

Problem D. Magic Multiplication

The multiplication table is printed by concatenating all one-digit products in a fixed order. The key point is that once the first digit a_1 of the first number is fixed, the whole second number is forced by parsing the first row from left to right.

Enumerate $a_1 \in \{1, \dots, 9\}$. While parsing the first row, each next product determines the corresponding digit b_i :

- If the next character is 0, then since $a_1 \neq 0$, the digit must be $b_i = 0$.
- Otherwise, among digits $0 \dots 9$, there is at most one value whose product with a_1 begins with the required digit sequence.

There is no ambiguity between one-digit and two-digit products: for a two-digit product to begin with digit x , it must be at least $10x$, which would require the other factor to be too large when the corresponding one-digit product is also possible.

After the whole B is known, use b_1 to parse all remaining rows and obtain A . If $b_1 = 0$, this reconstruction cannot determine the other rows and the candidate is rejected. Finally, rebuild the whole table from A and B and check that it matches the input string exactly.

A small table $factor[x][p]$, meaning “the digit that multiplied by x gives the printed product p ”, makes the parser simple. The enumeration has only nine candidates, so the running time is $\mathcal{O}(9|s|)$.

Problem E. Plants vs. Zombies

Binary search the answer. For a fixed target height x , decide whether all plants can reach at least x within the movement limit.

The greedy check works from left to right. Suppose the current accumulated height of plant i is cur_i . If $cur_i < x$, then the number of additional waterings needed at plant i is

$$t = \left\lceil \frac{x - cur_i}{a_i} \right\rceil.$$

Because water can only be applied after a move, the most efficient way to give these waterings before permanently moving right is to shuttle between plants i and $i + 1$. This costs $2t - 1$ moves and, during the shuttling, also waters plant $i + 1$ exactly $t - 1$ times. Therefore add $a_{i+1}(t - 1)$ to the next plant’s accumulated height.

If $cur_i \geq x$, no extra shuttling is needed for this plant. One still has to account for moving to the next useful position, but at the end of the array unnecessary final movement should not be charged. The check fails as soon as the required moves exceed m .

Feasibility is monotone in x , so binary search over $[0, m \max a_i]$. The greedy check is linear; the total time is $\mathcal{O}(n \log(m \max a_i))$.

Problem F. Tournament

The construction exists if and only if

$$k < \text{lowbit}(n),$$

where $\text{lowbit}(n)$ is the lowest set bit of n . Equivalently, $k \leq \text{lowbit}(n) - 1$.

Number players from 0 to $n - 1$. In round i , for $1 \leq i \leq k$, pair player j with

$$j \oplus i.$$

If the output is 1-indexed, print $(j \oplus i) + 1$.

This is a valid matching because xor is an involution:

$$(j \oplus i) \oplus i = j.$$

Also $i \neq 0$, so no player is paired with itself. For every $i < \text{lowbit}(n)$, the operation $j \mapsto j \oplus i$ stays inside the range $[0, n - 1]$. The necessity of the bound follows from the same combinatorial property used in IMO 2006 C5.

If $k \geq \text{lowbit}(n)$, print Impossible; otherwise output all k rounds. The time complexity is $\mathcal{O}(nk)$.

Problem G. Repair the Artwork

Treat every position as one of three types. Type 1 is forbidden: a chosen interval must not contain it. Type 2 is required: it must be covered by at least one of the chosen intervals. Type 0 has no restriction.

If the array contains only 0 and 1, let K be the number of intervals that contain no 1. Since each of the m intervals can be chosen independently, the answer is K^m .

Now handle type 2 positions by inclusion-exclusion. For each required position, subtract the cases where it is not covered. After this transformation, each term is again a 0/1 problem, and the contribution has the form

$$(-1)^c K^m,$$

where c is the number of required positions chosen by inclusion-exclusion.

Instead of enumerating all type 2 subsets, use dynamic programming over the positions that are treated as “1-like”. Add sentinels at positions 0 and $n + 1$. If the previous 1-like position is p and the next is i , then the zero block between them contributes

$$\frac{(i - p)(i - p - 1)}{2}$$

valid intervals. A type 1 position must be chosen as 1-like. A type 2 position may either be skipped or chosen as 1-like, and choosing it flips the inclusion-exclusion parity. A type 0 position is skipped.

Let $dp[pos][cnt][parity]$ store the number of ways after the last selected 1-like position is pos , the current number of valid intervals is cnt , and the parity is $parity$. Transition to the next selected 1-like position, add the block contribution above, and update the parity according to whether the new position was a selected type 2.

After the final sentinel, sum

$$(\# \text{even choices} - \# \text{odd choices}) \cdot cnt^m$$

over all possible cnt . The parity contributed by original fixed type 1 positions is handled as a global sign. The official implementation uses $\mathcal{O}(n^4)$ time with a small constant.

Problem H. Mirror

First handle several special cases. If the straight segment from the start to the destination is unobstructed, the answer is

$$(2m - 1) \cdot dist(S, T).$$

If the segment is blocked and $m = 1$, it suffices to run shortest path on the visibility graph built from the start, the destination, obstacle vertices, and mirror endpoints. If $m > 1$ and either endpoint lies to the left of the mirror, the task is impossible in the blocked case.

In the remaining case, $m > 1$, the direct path is blocked, and both endpoints are to the right of the mirror. The optimal cost separates into three independent shortest paths:

$$answer = d_S + (2m - 3)d_{S,T} + d_T.$$

Here d_S is the shortest path whose every point can see the start, d_T is the analogous path for the destination, and $d_{S,T}$ is the shortest path whose every point can see both.

Visibility changes only when crossing certain lines. It is enough to consider lines of the following forms:

- an endpoint and an obstacle vertex;
- an endpoint and a mirror endpoint;
- the mirror image of an endpoint and a mirror endpoint;
- the mirror image of an endpoint and an obstacle vertex;
- the mirror image of an endpoint and the mirror image of an obstacle vertex.

The obstacle boundaries and the mirror itself are not added as change lines because legal paths cannot cross them.

Build a graph as follows. Add all intersections of the change lines, discard points on or to the left of the mirror, and add S and T . For every pair of candidate points, check whether the segment crosses an obstacle

or the mirror. Then determine whether all points on the segment satisfy the required visibility condition. Since a segment can cross several change lines, sort the intersection points on the segment and test one sample point from each subsegment.

Run Dijkstra three times, once for each visibility requirement: start-only, both endpoints, and destination-only. In the official bounds, the remaining candidate points are at most 161 and the number of change lines is 26, so the total work is small; the stated complexity is about $\mathcal{O}(N^2T + N^2 \log N)$.

Problem I. Soldier Game

Reinterpret the game as covering the whole sequence by segments of length 1 or 2. The value of a segment is the sum of the elements it covers. We want to minimize

$$\max(\text{chosen segment values}) - \min(\text{chosen segment values}).$$

For a fixed lower bound v , every segment with value smaller than v is forbidden. Among the remaining segments, compute the minimum possible upper bound. Sweeping all possible values of v gives the answer.

Use a segment tree DP. For an interval $[l, r]$, keep

$$f(l, r, x, y), \quad x, y \in \{0, 1\}.$$

This is the minimum possible maximum segment value needed to cover $[l, r]$, where $x = 1$ means a_l is covered by a length-2 segment starting outside the interval on the left, and $y = 1$ means a_r is covered by a length-2 segment ending outside on the right. If $[l, r]$ is split at mid , the boundary state $k \in \{0, 1\}$ must agree on both sides, so

$$f(l, r, x, y) = \min_{k \in \{0, 1\}} \max\{f(l, mid, x, k), f(mid + 1, r, k, y)\}.$$

All candidate segment values are the n singleton values a_i and the $n - 1$ pair values $a_i + a_{i+1}$. Sort them increasingly. As the lower bound passes a value, update the corresponding leaf or adjacent-pair transition to infinity. The root value $f(1, n, 0, 0)$ is the best upper bound for the current lower bound.

The answer is the minimum of $f(1, n, 0, 0) - v$ during the sweep. The time complexity is $\mathcal{O}(n \log n)$.

Problem J. Books

A binary search on the amount of money is invalid, because the number of books bought is not monotone. For example, prices 4, 1, 2 allow buying two books with 3 units of money, one book with 4, and two books with 5.

Books with price 0 must always be bought. Let z be the number of zero-price books. If $z > m$, it is impossible to buy exactly m books. If all books must be bought, the answer is Richman.

Otherwise remove all zero-price books. We now need to force exactly $m - z$ positive-price books. Take the first $m - z$ positive books in order and sum their prices. Among the remaining positive books, let p be the minimum price. Then the maximum amount of money that still buys exactly the desired number of books is

$$(\text{sum of the chosen } m - z \text{ positive books}) + p - 1.$$

With one more unit, the cheapest remaining book can also be bought, so the condition would fail.

The algorithm is a single scan to count zeros, sum the needed positive books, and find the minimum remaining positive price. The time complexity is $\mathcal{O}(n)$.

Problem K. Airdrop

Place the airdrop at the origin after translating the coordinate system. We examine how the number of surviving players changes as the origin's x -coordinate moves.

Two players can collide only if they have the same distance to the origin. Since every player first moves vertically and then horizontally, only players on the same side of the y -axis can collide. Players exactly on the y -axis do not collide with anyone by definition.

Consider a sweep of the origin from left to right and focus on players currently on the left side. Between two consecutive player x -coordinates, all distances on that side increase by the same amount, so the collision status does not change. When the sweep crosses a player's x -coordinate, at most two new players enter a fixed distance class: the ones with $y = d$ and $y = -d$. For each distance, after all collisions, a survivor remains if and only if the previous survivor count plus the number of new entrants is exactly 1.

Therefore only coordinates near player positions matter. For each player coordinate X_i , test

$$X_i - 1, \quad X_i, \quad X_i + 1.$$

Compress these candidates and sweep them in sorted order. Maintain distance classes lazily by a common bias value. Compute the number of left-side survivors $L(x)$, repeat symmetrically for right-side survivors $R(x)$, and add the number of players exactly on the current y -axis.

The core sweep is linear after sorting and coordinate compression, so the total complexity is $\mathcal{O}(n \log n)$.

Problem L. Sub-cycle Graph

A graph is valid if each connected component is either an isolated vertex or a simple path, except for the special case where all vertices form one simple cycle.

There are simple boundary cases. If $m = 0$, the empty graph is unique. If $m > n$, no graph is possible. If $m = n$, all vertices must form one cycle, and the number of labeled undirected cycles is

$$\frac{(n-1)!}{2}.$$

Now assume $1 \leq m < n$. The graph is a forest of paths. Since it has n vertices and m edges, it has $n - m$ connected components. Let i be the number of nontrivial path components. Then the number of isolated vertices is $n - m - i$.

For a fixed i :

1. choose the isolated vertices: $\binom{n}{n-m-i}$;
2. among the remaining $m + i$ vertices, choose the $2i$ path endpoints: $\binom{m+i}{2i}$;
3. pair those endpoints into i unordered endpoint pairs: $(2i-1)!! = \prod_{k=1}^i (2k-1)$;
4. arrange the remaining $m - i$ internal vertices: $(m-i)!$;
5. split the internal-vertex sequence into i paths: $\binom{m-1}{i-1}$.

Thus the answer is

$$\sum_{i=1}^{n-m} \binom{n}{n-m-i} (m-i)! \binom{m-1}{i-1} \binom{m+i}{2i} (2i-1)!!.$$

Invalid binomial terms are treated as zero. Precompute factorials and inverse factorials modulo the given modulus; then the summation is linear. The time after preprocessing is $\mathcal{O}(n)$.

Problem M. Function and Function

The function replaces every decimal digit by a small contribution and sums the contributions. The digit contribution array is

$$[1, 0, 0, 0, 1, 0, 1, 0, 2, 1]$$

for digits $0, 1, \dots, 9$.

For $x \geq 2$, one application quickly reduces the value to about the number of decimal digits of x , so only a few direct simulations are needed even if x is large. Once the value becomes 0 or 1, the function alternates between these two values, so the remaining number of applications matters only by parity.

The algorithm is therefore:

1. while $K > 0$ and $x > 1$, replace x by the digit-contribution sum and decrease K ;
2. if $K = 0$, output x ;
3. otherwise output $x \oplus 1$ if K is odd, and x if K is even.

The number of direct applications is $\mathcal{O}(\log x)$, and after reaching 0/1 the answer is constant time.

Source. This is an unofficial English rewrite based on the SUA Wiki editorials for the 2018 ICPC Asia Qingdao Regional Contest: <https://sua.ac/wiki/2018-icpc-qingdao/>